

72.27 - Sistemas de Inteligencia Artificial

Resumen



1Q - 2026

Índice

Unidad 1 - Introducción.....	3
Unidad 2 - Agentes y Ambientes.....	4
Unidad 3 - Métodos de Búsqueda.....	5
Unidad 4 - Algoritmos de Mejoramiento Iterativo.....	11
Unidad 5 - Aprendizaje por Refuerzo.....	13
Unidad 6 - Algoritmos Genéticos.....	17
Unidad 7 - Aprendizaje Automático.....	23
Unidad 8 - Aprendizaje Supervisado.....	26
Unidad 9 - Aprendizaje No Supervisado.....	35
Unidad 10 - Deep Learning.....	40
Unidad 11 - Transformers.....	49
Unidad 12 - Redes Generativas Adversarias.....	52

Unidad 1 - Introducción

La IA surge de manera directa en el propio inicio de la materialización de las computadoras digitales en la Segunda Guerra Mundial. Existían tres necesidades que requerían el uso de computación:

- Criptoanálisis: descifrado del código enigma.
- Control Remoto de Torpedos
- Señales de Radars

Se ha englobado en **aprendizaje automático** un conjunto de métodos y algoritmos que se valen desde el teorema de Bayes (siglo XVII) hasta los árboles de decisión pasando por métodos de programación cuadrática, regresión y métodos de clasificación clásicos. Podemos establecer la siguiente definición: el aprendizaje automático son técnicas para identificar relaciones entre datos (X) o mapeos ($X \rightarrow Y$) basadas en algoritmos que, dada una serie de parámetros libres que estructuran esas relaciones o mapeos, pueden ajustarse mediante algún proceso de optimización matemática.

Así, llegamos a **Deep Learning**, que aparece como el resultado de la convergencia de distintos motivos. La posibilidad de disponer procesadores cada vez más poderosos, expresados no sólo por su velocidad y su memoria sino fundamentalmente por la cantidad de núcleos promovió la intensificación del uso y estudio en redes neuronales artificiales. La diferencia está en que los algoritmos que existían en los 80s eran fácilmente paralelizables, y con el surgimiento de las GPUs pudieron desarrollarse.

Una definición posible para IA entonces es:

- Un sistema que sensa, procesa y actúa de manera que es adecuada en un contexto físico y temporal particular.
- Requiere una corporización para manifestarse con claridad.
- Es un continuo, un "moving goalpost".
- Autonomía: como comportamiento emergente.
- Conciencia: manifiesta comportamientos que representan un conocimiento sobre su propia existencia, en relación al contexto físico y temporal, y una diferenciación en relación a los otros.

Unidad 2 - Agentes y Ambientes

El **agente** es entonces todo aquello que puede considerarse que percibe y responde o actúa en un ambiente, de forma independiente (autónomo). Tenemos distintos tipos. Es **racional** si realiza aquello que le permite obtener un mejor desempeño en su función. Es **omnisciente** si en todo momento, sabe el resultado real que producirán sus acciones para toda la posteridad. Es decir, un agente racional podría saber el efecto de sus acciones a corto plazo, pero si no tiene en cuenta cada uno de los escenarios posibles, no estamos hablando de un agente omnisciente.

Una tercera clasificación de agentes es según si tienen estado interno o no. Los que no tienen, también se los llama **reactivos** o **de reflejos simples**, y $\text{Acción} = f(\text{Percepción})$. Los que sí tienen, son llamados **basados en modelos**, y con cada percepción y/o acción pueden alterar su estado interno. $\text{Acción} = f(\text{Percepción}, \text{Estado Interno})$.

Un **ambiente** es un conjunto de componentes que conforman al entorno del problema. Pueden contener uno o más agentes. Aquí también podemos diferenciar distintos tipos. Notemos que un ambiente puede ser:

- Totalmente o Parcialmente Observable
- Determinístico o Estocástico (vs Estratégico)
- Episódico o Secuencial
- Estático o Dinámico (vs Semidinámico)
- Discreto o Continuo
- Conocido o Desconocido
- Individual o Multiagente
- Adversarial o Colaborativo

Unidad 3 - Métodos de Búsqueda

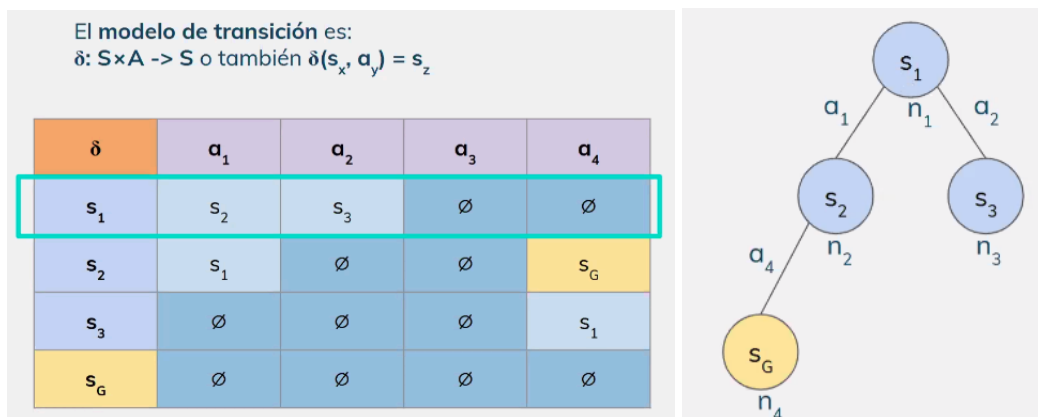
Antes de comenzar, debemos establecer las siguientes definiciones:

- **Estado:** Conjunto de valores del ambiente en un momento dado.
- **Acción:** Una función que toma un estado y retorna otro estado, si es aplicable.
- **Espacio de estados:** Conjunto de estados que incluye todos los estados alcanzables desde el estado inicial.
- **Espacio de acciones:** Conjunto de acciones que incluye todas las acciones aplicables a por lo menos un estado en el espacio de estados.

Para definir correctamente un problema, se debe tener:

- Estado Inicial: Estado en el cual, generalmente, el problema no está resuelto.
- Conjunto de posibles acciones: Dado un estado particular S , retorna el conjunto de acciones que pueden ejecutarse en S .
- Modelo de Transición: Describe el efecto de aplicar A (acción) a S (estado).
- Función de costo: Determina el costo para el modelo de transición.
- Condición de solución: Función que nos indica si un estado S es solución.

Podemos ver el modelo de transición como una función o una tabla:



Paralelamente, se tiene la tabla de costo:

$$g: S \times A \rightarrow \mathbb{R} \text{ o también } g(s_x, a_y) = w$$

Para poder armar una **representación del problema**, debemos establecer una definición de arquitectura de estados estratégicamente, entendiendo los operadores entre ellos y de forma tal que representen todo el espacio de estados.

Para la definición de acciones, se debe tener en cuenta las suposiciones de la descripción informal del problema. Definimos como **meta-acciones** a un conjunto de acciones que se ejecutan en seguidillas. Por ejemplo, moverse en diagonal - moverse horizontal y luego verticalmente.

Tenemos distintas estrategias de búsqueda. Es **completa** si encuentra una solución (de existir y ser alcanzable), y es **óptima** si encuentra la solución alcanzable con menor costo. Las **búsquedas desinformadas** no tienen más información de los estados que los que otorga la definición del problema. Sólo se distinguen estados "goal" de los que no lo son. Son poco eficientes en problemas donde podemos realizar estimaciones. Algunos algoritmos de este tipo de búsqueda son:

- BFS
- DFS
- IDDFS
- Costo Uniforme

Las **búsquedas informadas** tienen más información para comparar entre dos estados cuál es "mejor". Pueden estimar cuánto les falta para solucionar el problema, y la estimación a la solución desde un estado se llama **heurística**. Pueden estimar cuál es el costo del problema. En problemas complejos, suelen ser más eficientes que los métodos no informados. Algunos de ellos son:

- Greedy
- A*

Un estado es **expandido** cuando dentro del método de búsqueda se le han aplicado todas las acciones aplicables, es decir que está explorado. En el caso de la derecha, únicamente el nodo 1 es el expandido. Es importante no confundir nodo con estado. Un nodo contiene un estado. De esta forma, un estado puede estar representado en diferentes nodos. Siendo $g(n)$ el costo acumulado del nodo (la suma del arco desde 1, hasta n), podemos encontrar dos nodos n_1 , y n_2 , que pueden contener al mismo estado, de forma tal que $g(n_1) \neq g(n_2)$.

El primer algoritmo es **Breadth First Search**, que expande los nodos de menor profundidad primero y encuentra la solución con menor profundidad. Es completo si

el factor de ramificación es finito. Es Óptimo cuando el problema es de costo uniforme, es decir que las acciones tienen todas el mismo costo.

El algoritmo **Uniform Cost Search** expande el nodo con menor costo acumulado. Es idéntico a BFS en problemas de costo uniforme. Es óptimo si $g(\text{sucesor}(n)) \geq g(n) \forall n$; es decir $g(a) \geq 0$; $\forall a$ acción (si el costo de una acción es < 0 , no es óptima).

El algoritmo **Depth First Search** expande el nodo de mayor profundidad. Suele ser de menor complejidad espacial, sobre todo en su versión sin estados repetidos (más adelante). Suele ser más rápido que BFS cuando existen muchas soluciones. No es óptimo.

El algoritmo **Depth Limited Search** es igual que DFS, sólo que se limita la profundidad. Es decir que establecemos un límite de profundidad, por ejemplo debido a la capacidad de cómputo. No es completa ni óptima.

El algoritmo **Iterative Deepening Depth First Search** es una aplicación de DLS donde la profundidad varía según cierto criterio. Se puede implementar manteniendo los nodos hoja que no fueron expandidos por cota de profundidad entre cada iteración. Por ejemplo, podría configurarse con un profundidad inicial de 1 y creciente en 1 en cada iteración - sería un BFS (más costoso si no mantiene el árbol al reiniciar).

Los **Estados Repetidos** se basan en no regresar a estados ancestros para no crear rutas que tengan ciclos y no volver a expandir estados ya expandidos previamente. Es muy útil para DFS y BFS, pero se debe tener cuidado con DLS e IDDFS, pues si existe una solución a profundidad N, podemos no encontrarla. Hay que contemplar el compromiso vs el costo de almacenar y chequear estados.

Una **función heurística** estima el costo de la ruta más barata desde el estado e hacia un estado meta. Tomamos $g(a)$ como costo de la acción a. Si e es un estado objetivo, $h(e) = 0$. Si e no es un estado objetivo:

- Si $g(a) > 0$, $\forall a$: Acción $\Rightarrow h(e) > 0$
- Si $g(a) \geq 0$, $\forall a$: Acción $\Rightarrow h(e) \geq 0$

Una heurística es **admisible** si nunca sobreestima el costo real ($h(a) \leq \text{costo a la solución}$, para todo a). La heurística **perfecta** (que estima el costo real) se denomina heurística estrella : $h^*(n)$.

La **combinación de heurísticas** se define de la siguiente manera. Dado un conjunto de heurísticas admisibles $h_1, \dots, h_m$; se puede definir como una nueva heurística h' a la combinación de ellas, definida de la siguiente forma:

$$h' = \text{Max}(h_1, \dots, h_m)$$

Esta nueva heurística tiene como propiedades:

- Es admisible.
- Domina a todas las heurísticas que la conforman.

El primer algoritmo de búsqueda informada es el **local greedy search**. Se comienza con el nodo raíz y se expande, y de los recién expandidos se calcula h . Luego, se toma el nodo con menor h y se expande. Si el conjunto recientemente expandido es vacío, se realiza backtracking. Este algoritmo no es óptimo y no es completo si no evalúa repetición de estados. Una buena función heurística reduce significativamente su complejidad temporal y espacial. Opera siempre sobre un conjunto acotado, lo que la hace muy veloz.

El **global greedy search** es una generalización del anterior. Se utiliza el Algoritmo de búsqueda genérico (DFS, BFS, IDDFS, etc.), utilizando $h(n)$ como ordenamiento de Fr. Nuevamente, no es óptimo y no es completo si no evalúa repetición de estados. También, una buena función heurística reduce significativamente su complejidad temporal y espacial. Es un poco más costosa que LGS pero logra un backtracking más inteligente.

El algoritmo **A*** es uno de los más importantes. Se basa en la función:

$$f(n) = g(n) + h(n) \mid \text{costo} + \text{heurística}$$

El algoritmo ordena a los nodos en frontera según esta función. Si dos nodos tienen igual $f(n)$, se elige el nodo con menor $h(n)$. Lo interesante es que si $h(n)$ es admisible $\Rightarrow f(n)$ nunca sobreestima el costo real de la mejor solución que pasa por el nodo n , es decir que nos da una solución óptima. Además, es completa si hay una ramificación finita y el costo es mayor que un $\epsilon > 0$, y es óptima si los costos de los arcos son mayores a 0. Requiere memoria y procesamiento de heurísticas. A* con

una heurística h_1 se denomina A^*_1 . Para que A^* sea óptima, cada nodo del grafo debe tener un número finito de sucesores. El costo de cada arista debe ser mayor que un $\varepsilon > 0$ y la heurística debe ser admisible.

$$h(n) \leq h^*(n)$$

Estas tres condiciones garantizan que A^* encuentre el camino óptimo a la solución, si existe. Se define $f^*(n) = g(n) + h^*(n)$.

Tomemos ahora el concepto de **dominancia**. M_1 domina a M_2 si cada nodo expandido por M_1 también es expandido por M_2 . Por otro lado, M_1 **domina estrictamente** a M_2 si M_1 domina a M_2 y M_2 no domina a M_1 .

La **eficiencia de heurísticas** funciona de la siguiente manera. Suponiendo h_1 y h_2 heurísticas admisibles, si h_2 domina a $h_1 \Rightarrow A^*_2$ expandirá menos nodos que A^*_1 . ¿Por qué? Existirán nodos que A^*_1 expandió (y no llevan al camino óptimo) y A^*_2 no, por lo que, para esos nodos:

$$g(n) + h_1(n) \leq f^*(n) \leq g(n) + h_2(n)$$

Definimos la **consistencia de heurísticas** de la siguiente forma. Sean n_i y n_j dos nodos en un grafo de búsqueda tal que n_j es vecino de n_i ; y sea $c(a, b)$ el costo de la acción desde a hasta b .

- $h(n_i) \leq c(n_i, n_j) + h(n_j)$
- $h(n_g) = 0$

Para el algoritmo A^* tenemos variaciones, como **Iterative Deepening A^* (IDA*)**. Está inspirado en IDDFS, y se realiza un corte iterativo con un límite. También aquí se utiliza DFS. Esta búsqueda es completa y óptima bajo las mismas condiciones que A^* . Requiere menos memoria que A^* , pero puede expandir más nodos e incluso expandir muchas veces el mismo nodo (mismo problema que IDDFS).

Otro algoritmo es el **Heuristic Path Algorithm (HPA)**. Se realiza mediante Best-First con $f(n) = (1-w)*g(n) + w*h(n)$. Notemos que:

- $w = 0 \Rightarrow$ Uniform Cost Search
- $w = 1/2 \Rightarrow A^*$ Search
- $w = 1 \Rightarrow$ Global Greedy Search

Algunas de las técnicas utilizadas para encontrar heurísticas son:

- Dividir al problema en sub-problemas.
- Relajar las limitaciones/reglas del problema.
- Tomar una heurística como el máximo de otras.
- Mezclar heurísticas, contemplando la valuación de cada una de ellas, sobre todo cuando atacan diferentes sub-problemas.

Unidad 4 - Algoritmos de Mejoramiento Iterativo

Un **algoritmo de mejoramiento iterativo** parte de una solución y busca (mediante perturbaciones a mi solución inicial) una solución mejor en cada paso. El propio algoritmo es el encargado de ir mejorando la solución. Lo hace en cada paso, no en cada corrida.

Para hallar una solución óptima, podemos realizar **cambios azarosos** en nuestra solución original que nos den como resultado una mejor solución. Es importante definir qué es un cambio azaroso en el problema particular que estoy queriendo optimizar, para lo que necesitamos definir un entorno de soluciones.

Hill Climbing es un algoritmo de mejoramiento iterativo que mantiene una sola solución actual y la va mejorando con cambios pequeños. Se parte de una solución candidata s y, en cada paso, se miran sus vecinos (soluciones obtenibles con un cambio pequeño) y se mueve al vecino que mejora el valor de una función de evaluación $f(s)$. El problema clave es que, como solo mira mejoras locales, se puede estancar en situaciones donde ya no hay un vecino mejor aunque no esté en el mejor valor global: un máximo local (un pico que no es el más alto), un máximo local plano (una meseta donde muchos vecinos tienen el mismo valor y no hay “señal” para elegir), o incluso una ensilladura (un punto que parece bueno en una dirección pero tiene salida en otra). Existen distintos tipos de hill climbing.

- **Simple Hill Climbing:** Elige la primera solución de la frontera con mejor valuación que el estado actual.
- **Steepest Hill Climbing:** Siempre elige la solución de la frontera con mejor valuación que el estado actual.
- **Stochastic Hill Climbing:** Elige al azar una solución de la frontera y, en base a que tan buena es, decide si moverse o elegir otra.

Simulated Annealing es una variante de Hill Climbing pensada para evitar quedar atrapado en máximos locales. La idea central es que, además de aceptar movimientos que mejoran $f(s)$, a veces también acepta movimientos que empeoran (bajar en el gráfico), pero lo hace de manera controlada: al principio es “más permisivo” y con el tiempo se vuelve “más estricto”. Entonces cada vez que toma un

nodo de Frontera, ignora su valuación con una probabilidad relativa a la temperatura (y, opcionalmente, también proporcional a su valuación).

Beam Search es una estrategia de búsqueda que combina ideas de búsqueda por niveles con una “poda” fuerte para mantener el costo bajo. En vez de seguir una sola solución como Hill Climbing, mantiene varios candidatos a la vez (un “haz” o beam). En cada iteración, expande esos candidatos generando sucesores, evalúa qué tan buenos son con una función $f(s)$, y se queda solo con los k mejores para continuar al siguiente paso. Ese k es el **beam width** y controla el trade-off: más grande explora más pero cuesta más, más chico es más rápido pero más riesgoso.

Stochastic Beam Search es muy parecido a Beam Search, ya que sigue manteniendo un conjunto de k candidatos y, en cada iteración, generás muchos sucesores y necesitás elegir cuáles k sobreviven para el próximo paso. La diferencia clave es cómo se eligen esos k . Beam Search “clásico” toma determinísticamente los k con mejor evaluación, mientras que Stochastic Beam Search los selecciona al azar, pero con una probabilidad proporcional a su valoración $f(s)$. O sea, cuanto mejor es un candidato, más chances tiene de ser elegido, pero no es seguro. Es similar al proceso de selección natural.

Los **algoritmos genéticos** son una familia de métodos de búsqueda/optimización inspirados en la evolución natural. En vez de tener una sola solución, trabajan con una población de soluciones candidatas, donde cada solución (estado) se interpreta como un “individuo”. A cada individuo se le asigna una medida de calidad llamada **aptitud** (fitness), que básicamente es la función que dice qué tan buena es esa solución para el problema.

Unidad 5 - Aprendizaje por Refuerzo

Existen tres tipos de aprendizaje automático. En **aprendizaje supervisado**, el modelo aprende a partir de ejemplos donde ya conocemos la respuesta correcta: cada dato viene con una etiqueta (por ejemplo, mails marcados como “spam/no spam” o fotos con “gato/perro”). El objetivo es que el sistema encuentre una función que, dado un nuevo ejemplo, pueda predecir su etiqueta generalizando lo aprendido.

En **aprendizaje no supervisado**, en cambio, no hay etiquetas; el modelo solo ve los datos “en bruto” y trata de descubrir estructura o patrones internos. Eso suele significar agrupar ejemplos similares (clustering), reducir dimensionalidad para encontrar representaciones más simples, o detectar outliers.

Por último, el **aprendizaje por refuerzo** aprende a partir de la interacción con un entorno: un agente toma acciones, el entorno cambia y el agente recibe una recompensa (o castigo). La señal de aprendizaje no es una etiqueta por ejemplo, sino un feedback que puede llegar tarde, porque una acción puede ser mala o buena según su efecto acumulado en el tiempo. El objetivo es aprender una política de decisiones que maximice la recompensa total a largo plazo, como en juegos, robótica o control.

Un **MDP (Proceso de Decisión de Markov)** es el modelo matemático estándar para describir problemas de decisión secuenciales en aprendizaje por refuerzo. Se define como (S, A, R, p, γ) : S es el conjunto finito de estados posibles del entorno; A es el conjunto finito de acciones que el agente puede tomar; y R es el conjunto (finito) de recompensas posibles. La idea es que el agente está en algún estado s , elige una acción a , y el entorno responde con una recompensa r y un nuevo estado s' . La dinámica del entorno se captura con p , que aparece como una probabilidad conjunta: $p(s', r | s, a)$. Eso significa: estando en s y haciendo a , cuál es la probabilidad de terminar en el estado siguiente s' y recibir recompensa r . El último parámetro, $\gamma \in [0, 1]$, es el factor de descuento: controla cuánto valen las recompensas futuras respecto de las inmediatas. Si γ es cercano a 0, el agente es muy “cortoplacista”; si es cercano a 1, planifica más a largo plazo.

La **propiedad de Markov** es el supuesto clave: para predecir lo que pasa después, solo importa el estado actual y la acción actual, no toda la historia previa. Formalmente, la probabilidad de (s', r) dada toda la secuencia pasada es igual a la probabilidad condicionada sólo por (s_t, a_t) . Esto es lo que hace que el problema sea tratable: si elegís bien cómo se define el estado S_t , ese estado ya “resume” toda la información relevante del pasado.

El agente toma decisiones siguiendo una **política** — una regla que, dado el estado actual, asigna una probabilidad a cada acción posible.

$$\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1] \quad \sum_{a \in \mathcal{A}} \pi(a | s) = 1 \quad \forall s \in \mathcal{S}$$

Puede ser:

- Determinista: asigna probabilidad 1 a una sola acción por estado
- Estocástica: distribuye probabilidad entre varias acciones — útil durante la exploración

Dado que el agente sigue π , ¿cuán bueno es estar en un estado o tomar una acción? Esta pregunta nos permite comparar políticas. Para esto, tomamos las **funciones de valor**. La V representa el valor esperado.

Valor de estado $v_\pi : \mathcal{S} \rightarrow \mathbb{R}$

$$v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$$

Valor de acción $q_\pi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$$

Ambas funciones satisfacen una condición de consistencia recursiva — el valor actual es la recompensa inmediata esperada más el valor descontado del paso siguiente. Podemos ahora definir las **ecuaciones de Bellman**. Para v_π — promedio sobre mi acción, luego sobre la respuesta del mundo:

$$v_\pi(s) = \sum_a \underbrace{\pi(a|s)}_{\text{mi acción}} \sum_{s', r} \underbrace{p(s', r|s, a)}_{\text{respuesta del mundo}} [r + \gamma v_\pi(s')]$$

Para q_π — promedio sobre la respuesta del mundo, luego sobre mi siguiente acción:

$$q_{\pi}(s, a) = \sum_{s', r} \underbrace{p(s', r | s, a)}_{\text{respuesta del mundo}} \left[r + \gamma \sum_{a'} \underbrace{\pi(a' | s')}_{\text{siguiente acción}} q_{\pi}(s', a') \right]$$

La idea de una **Generalized Policy Iteration (GPI)** es que siempre se alterna entre dos procesos que se retroalimentan. En evaluación, se toma una política π fija (una regla de qué acción elegir en cada estado) y se calcula qué tan buena es, es decir su función de valor v_{π} (valor por estado) o q_{π} (valor por estado-acción). Esa función de valor dice la recompensa esperada a largo plazo si se sigue comportando según π . Luego viene la actualización (o mejora): usando v_{π} o q_{π} , se construye una política nueva π' que sea mejor que la anterior, típicamente eligiendo en cada estado la acción que maximiza el valor esperado (intuición “greedy” respecto al valor). Esa nueva política vuelve a entrar al paso de evaluación, y el ciclo se repite: evaluación produce funciones de valor, y actualización produce una política mejor. Si se cumplen ciertas condiciones, este proceso converge hacia π^* .

Los algoritmos difieren en cómo hacen cada paso — no en la estructura. Veamos por qué converge a π^* . Tenemos el **teorema de mejora de política**: si π' es greedy respecto a q_{π} , entonces π' es al menos tan buena como π :

$$v_{\pi'}(s) \geq v_{\pi}(s) \quad \forall s \in \mathcal{S}$$

En **programación dinámica** aplicada a MDPs se parte de un supuesto fuerte: la dinámica del entorno es completamente conocida. Eso significa que, para cualquier estado y acción, se conoce la distribución de probabilidad de los posibles estados siguientes y recompensas. Con ese modelo explícito, no hace falta aprender por prueba y error; se puede calcular de forma exacta qué conviene hacer mediante expectativas.

Cuando la dinámica $p(s', r | s, a)$ no es conocida, ya no se puede hacer programación dinámica “sumando” sobre todas las transiciones posibles, y se reemplaza ese cálculo exacto por experiencia real obtenida al interactuar con el entorno. La idea de **Monte Carlo** en aprendizaje por refuerzo es aprender valores (típicamente $Q(s, a)$ o $V(s)$) mirando episodios completos: el agente ejecuta una política π , observa la secuencia de estados, acciones y recompensas, y con eso estima qué tan buena fue

cada decisión en promedio. El punto clave es que la actualización se hace usando el retorno observado G_t , que es la suma descontada de recompensas desde un tiempo t hasta el final del episodio. Como no se conoce el “valor esperado” teórico, se usa el retorno real como una muestra; repitiendo muchos episodios, el promedio de esos retornos converge al valor esperado gracias a la ley de los grandes números. Por eso Monte Carlo es conceptualmente simple: aprende promediando resultados reales, sin modelo.

Q-learning propone aprender valores de acción $Q(s,a)$ sin tener que esperar a que termine un episodio. La idea es que el agente, mientras interactúa, actualiza su estimación en cada paso usando la información disponible hasta ese momento. En lugar de calcular el retorno completo G_t mirando todo el futuro (como en Monte Carlo), usa una aproximación “de un paso” que combina recompensa inmediata y una estimación del mejor futuro posible. Esa aproximación es el bootstrap: se toma como objetivo algo como “lo que cobro ahora” más “lo mejor que creo que puedo obtener después”. La consecuencia importante es que Q-learning puede trabajar en tareas continuas y aprende online, pero introduce un trade-off: al bootstrappear, aprende con un objetivo que también está siendo aprendido, lo cual puede generar más varianza o inestabilidad si no se maneja bien.

Unidad 6 - Algoritmos Genéticos

Un **algoritmo genético** se entiende como un método de optimización y adaptabilidad inspirado en la selección natural. En vez de construir un plan exacto para llegar a una meta, se trabaja con una función de aptitud que mide qué tan buena es cada solución, y se intenta mejorar esa aptitud a lo largo del tiempo. La mecánica se organiza en generaciones. Se parte de una población inicial de soluciones y luego se repite un ciclo: primero se hace recombinación, mezclando características de “padres” para crear “hijos”; después se aplica mutación, introduciendo cambios aleatorios pequeños para mantener diversidad y explorar regiones nuevas; y luego se realiza selección según aptitud, donde las soluciones con mejor desempeño tienen más probabilidad de sobrevivir y reproducirse. El resultado es una nueva población que, en promedio, tiende a mejorar.

En un algoritmo genético, el primer paso consiste en definir cómo se codifican las soluciones, es decir, establecer su **genotipo**. Esta puede tomar distintas formas, como cadenas de bits, vectores de valores reales, permutaciones o incluso estructuras más complejas como árboles. Tenemos luego los siguientes componentes:

- **Locus:** posición/índice dentro del cromosoma donde se guarda un “gen” (un atributo o parte de la solución).
- **Cromosoma:** la representación completa de una solución candidata (por ejemplo, una cadena de bits).
- **Fenotipo:** la solución “interpretada” en el dominio real; las características observables que resultan del cromosoma.
- **Alelos:** los valores posibles que puede tomar un locus (por ejemplo, 0/1 o combinaciones que codifican color/forma).

La **población inicial** es el punto de partida de un algoritmo genético: un conjunto de soluciones candidatas que el método va a ir mejorando generación tras generación. En general se busca que desde el inicio haya diversidad, porque si todos los individuos se parecen demasiado el algoritmo corre el riesgo de explorar sólo una parte del espacio y estancarse rápido.

La **función de fitness o aptitud** es una función de evaluación de los individuos. Representa que tan bien está un individuo en la evaluación del problema. Mientras más alto sea su resultado, más adaptado está dicho individuo al problema. El **fitness relativo** pone ese resultado en relación al resto. Es decir, ya no solo dice que tan bueno es, sino que tan bueno es comparado con el resto. Se define:

$$p(i) = \frac{f(i)}{\sum_{j=1}^N f(j)} \quad f(i) : \text{Fitness}$$

La **selección** define quiénes pasan a reproducirse o a sobrevivir hacia la próxima generación. La idea general es balancear dos fuerzas: por un lado, favorecer a los individuos con mayor aptitud para que la población mejore; por el otro, mantener algo de diversidad para no quedarse atrapado temprano en soluciones “buenas pero no las mejores”.

- El **muestreo directo o elitismo** toma directamente a los individuos con mejor aptitud. Esto acelera la mejora porque preserva lo mejor encontrado, pero si se usa en exceso puede hacer que la población se vuelva muy parecida rápidamente y se pierdan alternativas que todavía no tuvieron chance de madurar.
- El **muestreo aleatorio** ignora la aptitud y elige individuos al azar; mantiene diversidad, pero es poco eficiente para mejorar porque no aprovecha información sobre qué soluciones son mejores.
- El **muestreo estocástico** mezcla aptitud y azar: los individuos con mayor aptitud tienen más probabilidad de ser seleccionados, pero los peores no quedan completamente descartados. Este enfoque suele ser el más práctico porque introduce una presión selectiva razonable sin apagar la exploración, y ayuda a evitar que el algoritmo converja demasiado rápido a un óptimo local.
- La **selección por ruleta** es una forma de muestreo estocástico donde cada individuo tiene una probabilidad de ser elegido proporcional a su aptitud. Primero se transforma la aptitud de cada individuo en una probabilidad relativa p_i , típicamente normalizando para que todas sumen 1.
- La **selección universal** es una variante de la ruleta que busca que el muestreo sea más “parejo” y menos ruidoso. En vez de tirar K números aleatorios, se tira un solo número aleatorio r en $[0,1]$ y a partir de ahí se generan K puntos igualmente espaciados.

- La **selección por ranking**, en vez de usar directamente el fitness numérico, se ordena a los individuos según su aptitud y se trabaja con el orden relativo: quién es 1º, 2º, 3º, etc. Así la presión selectiva depende del ranking y no de escalas o outliers del fitness.
- La **selección entrópica o de Boltzmann** toma una idea parecida a “temperatura” para controlar cuán fuerte es la presión selectiva. En lugar de usar el fitness directamente como peso de ruleta, se construye una pseudo-aptitud transformándolo con una función exponencial: a cada individuo se le asigna un peso proporcional a $e^{f(i)/T}$.
- La **selección por torneos determinísticos** funciona como una “competencia” repetida para elegir padres o sobrevivientes. En vez de asignar probabilidades globales como en ruleta, se toman subgrupos pequeños de la población: de una población total de tamaño N , se eligen M individuos al azar y se los hace “competir” comparando sus aptitudes. Como el torneo es determinístico, el ganador es simplemente el de mayor fitness dentro de ese grupo.
- La **selección por torneos probabilísticos** se desarrolla de la siguiente manera:

1. Se elige un valor de **Threshold** en $[0.5, 1]$
2. De la población de tamaño N , se eligen **2** individuos al azar.
3. Se toma un valor r al azar uniformemente en $[0,1]$.
 - a. Si $r < \text{Threshold}$ se selecciona el más apto.
 - b. Caso contrario, se selecciona el menos apto.
4. Se repite el proceso (1.) hasta conseguir los K individuos que se precisan.

El **crossover (cruza)** es el operador que toma dos padres, los empareja y recombina sus genes para producir hijos. La intuición es que cada padre tiene partes útiles de la solución, y al mezclarlas se pueden construir descendientes que hereden lo mejor de ambos, explorando combinaciones nuevas sin empezar desde cero.

- En el **cruce de un punto** se toman dos padres representados como cadenas de genes y se elige al azar una posición de corte P (un locus). A partir de ese punto, se intercambia el “resto” de la cadena entre los padres.
- En el **cruce de dos puntos** se toma la misma idea del cruce de un punto, pero en lugar de cortar una sola vez, se eligen dos posiciones P_1 y P_2 (con $P_1 \leq P_2$). Eso divide el cromosoma en tres tramos: el prefijo antes de P_1 , el bloque del

medio entre P_1 y P_2 , y el sufijo después de P_2 . La recombinación consiste en intercambiar el tramo del medio entre los dos padres, manteniendo iguales los tramos de afuera.

- El **cruce anular** es una variante donde el cromosoma se trata como si fuera circular, como un anillo. En vez de elegir dos puntos fijos como en el cruce de dos puntos, se elige un punto de inicio P y una longitud L . A partir de P , se toma un segmento continuo de largo L y ese segmento se intercambia entre los dos padres para generar los hijos.
- En el **cruce uniforme** no se corta el cromosoma en uno o dos puntos, sino que se decide gen por gen de qué padre hereda cada posición. Para cada locus se hace un sorteo con una probabilidad P (típicamente 0.5): según el resultado, en esa posición se mantienen los alelos como estaban o se intercambian entre los padres.

Se define **mutación** como una variación en la información genética que se almacena en el cromosoma. Para que la mutación sea más eficiente, una correcta arquitectura y separación de genes es necesaria. Su motivación principal es enriquecer la diversidad genética de la población. Dado una probabilidad de mutación P_m :

- **Mutación de Gen:** Se altera un solo gen con una probabilidad P_m .
- **Mutación Multigen Limitada:** Se selecciona una cantidad $[1, M]$ (azarosa) de genes para mutar, con probabilidad P_m .
- **Mutación Multigen Uniforme:** Cada gen tiene una probabilidad P_m de ser mutado.
- **Mutación Completa:** Con una probabilidad P_m se mutan todos los genes del individuo, acorde a la función de mutación definida para cada gen.

También hace falta definir qué es una mutación sobre un alelo de un gen. Algunas posibilidades son:

- Tomar un nuevo alelo aleatorio sobre el gen.
- Aplicar un delta al alelo, en algún sentido y con alguna distribución.

Si la población converge en sus características, sin presentar más variedad de la que introduce la mutación y sin haber llegado a una aptitud aceptable, decimos que la población sufrió una **convergencia prematura**. Esto suele provocar que la solución del método arroje un subóptimo deficiente, a menos que dicho punto de convergencia sea el máximo global (difícil o imposible de analizar) ó que la

población escape de dicho máximo local por mutación (también, muy difícil de analizar).

La **supervivencia** en algoritmos genéticos es el mecanismo que define qué individuos pasan a formar parte de la siguiente generación después de haber evaluado aptitudes y generado descendencia mediante cruce y mutación. Su objetivo es mantener el tamaño de población constante mientras decide cuánta continuidad hay entre generaciones.

- La **supervivencia aditiva** describe una forma de armar la próxima generación donde los hijos no reemplazan automáticamente a los padres. En lugar de “tirar” a los padres y quedarse solo con los hijos, se construye un conjunto candidato más grande que incluye a todos los individuos actuales más los hijos.
- La **supervivencia exclusiva** describe un esquema donde, en la medida de lo posible, la nueva generación se arma solo con hijos, o sea que el reemplazo de la población es mucho más “duro” que en la supervivencia aditiva.
 - $K > N$: La nueva generación se genera seleccionando N de los K hijos exclusivamente.
 - $K \leq N$: La nueva generación se conformará por los K hijos generados + $(N-K)$ individuos seleccionados de la generación actual.

La **brecha generacional** es un parámetro $G \in [0,1]$ que indica qué fracción de la población se renueva en cada generación.

Los **criterios de corte** son las reglas que determinan cuándo dejar de iterar en un algoritmo genético. Una forma simple es cortar por tiempo o por cantidad de generaciones, que pone un límite fijo al esfuerzo. Otra opción es cortar cuando se alcanza una solución aceptable, es decir, cuando el fitness supera un umbral que ya cumple los requisitos del problema.

Un **esquema** es una forma de describir un patrón parcial que comparten varios cromosomas: en vez de fijar todos los genes, se fijan solo algunas posiciones y se deja el resto como “comodines”. Sirve para hablar de familias de soluciones que tienen ciertas partes iguales, sin importar cómo sean los genes restantes. El **orden de un esquema** mide cuánta información fija contiene ese patrón. Se define como la

longitud del esquema menos la cantidad de comodines: en otras palabras, es cuántas posiciones están especificadas (no son *). El **teorema de los esquemas** establece la evolución no “descubre” soluciones completas de golpe, sino que va amplificando patrones parciales que resultan útiles.

Unidad 7 - Aprendizaje Automático

El **aprendizaje automático** se define bajo un enfoque libre de modelo como la capacidad de determinar el mapeo o la relación funcional entre un conjunto de entradas X y sus respectivas salidas Y . Este proceso no depende de una estructura teórica rígida previa, sino que se basa en el ajuste de parámetros libres dentro del sistema. El objetivo es que, mediante la observación de datos, el sistema logre configurar estos parámetros de manera que pueda predecir o clasificar correctamente nuevas entradas basándose en patrones identificados previamente.

El aprendizaje mediante datos en Machine Learning se organiza en tres paradigmas fundamentales:

- El **aprendizaje supervisado** se basa en el entrenamiento con datos previamente etiquetados por un humano.
- El **aprendizaje no supervisado** busca identificar estructuras o patrones ocultos en los datos sin una guía externa.
- El **aprendizaje por refuerzo** se centra en que un agente aprenda a tomar decisiones óptimas mediante un sistema de recompensas y penalizaciones basadas en su interacción con un entorno.

El problema básico de la **optimización** es encontrar la combinación de los parámetros para una función que hacen que esta sea lo más grande o más chica posible. La optimización en ingeniería se clasifica según la naturaleza de las variables y la forma de la función objetivo.

- La **optimización discreta** se enfoca en problemas donde las soluciones deben ser números enteros o combinaciones específicas, abarcando la programación entera y el estudio de ecuaciones diofánticas.
- La **optimización lineal** aborda problemas con relaciones estrictamente proporcionales.
- La **optimización convexa** tiene una estructura matemática que permite garantizar matemáticamente el hallazgo de extremos globales, asegurando que el punto mínimo o máximo encontrado sea efectivamente la mejor solución posible para el sistema.

- La **optimización no convexa** presenta un aumento en la complejidad debido a la presencia de múltiples óptimos locales que pueden confundir al algoritmo, lo que requiere el uso de diferentes órdenes de métodos para su resolución.

El problema general de optimización matemática consiste en encontrar el conjunto de valores para las variables de diseño, representadas por el vector x que minimicen una función objetivo o de costo $f(x)$. Esta función actúa como el criterio principal para evaluar la calidad de una solución propuesta, transformando un espacio multidimensional de variables en un valor escalar que representa el rendimiento o el error del sistema.

La **programación lineal** es una técnica de optimización matemática donde tanto la función objetivo como las restricciones que delimitan el sistema son funciones lineales. El proceso busca encontrar los valores de las variables de decisión que maximizan o minimizan un criterio específico, cumpliendo estrictamente con un conjunto de desigualdades que representan limitaciones físicas, técnicas o económicas.

El **procedimiento general de optimización** es un proceso iterativo diseñado para aproximarse de manera sucesiva al valor mínimo de una función de costo. El algoritmo comienza en un punto inicial x_k , que actúa como la posición actual en el espacio de búsqueda. A partir de este punto, el sistema debe determinar una dirección de movimiento d_k que apunte hacia una zona de menor costo y calcular una longitud de paso α_k , la cual define qué tan grande será el desplazamiento en esa dirección. La actualización del sistema se realiza mediante la regla matemática $x_{k+1} = x_k + \alpha_k d_k$, donde el nuevo punto se convierte en la base para la siguiente iteración. Este ciclo se repite hasta que el algoritmo alcanza un criterio de parada, como cuando el cambio entre puntos es despreciable o se llega a un punto donde la pendiente es cercana a cero.

El **método del gradiente descendiente**, también conocido como de máximo descenso, es una de las técnicas de optimización de primer orden más fundamentales en el entrenamiento de modelos de inteligencia artificial. Su principio se basa en una propiedad matemática clave: el gradiente de una función, $\nabla f(x_k)$, señala la dirección de máximo crecimiento en un punto determinado. Por lo tanto,

para minimizar una función de costo, el algoritmo debe moverse en el sentido opuesto al gradiente, lo que se define formalmente como la dirección de búsqueda $d_k = -\nabla f(x_k)$. Este desplazamiento asegura que, en cada iteración, el sistema baje por la pendiente más pronunciada hacia el valor mínimo.

El **método del gradiente descendiente con momentum** surge como una solución técnica para mitigar el problema del zigzaguo u oscilaciones excesivas que presenta el algoritmo estándar en superficies de error con valles estrechos y empinados. Esta técnica introduce un término regularizador que consiste en tomar la dirección de descenso como una combinación lineal entre el gradiente actual y las direcciones calculadas en pasos anteriores. Al incorporar esta "memoria" del movimiento, el sistema logra amortiguar los cambios bruscos de dirección y acelerar la convergencia en el sentido correcto, emulando el comportamiento físico de una bola que adquiere velocidad mientras rueda hacia el fondo de una colina.

El **método de Newton** es una técnica de optimización de segundo orden que busca el mínimo de una función utilizando no solo la pendiente (gradiente), sino también su curvatura. Los **métodos Quasi-Newton** surgen como una solución técnica para mitigar el elevado costo computacional que implica el método de Newton tradicional. En lugar de realizar este cálculo exacto, estos algoritmos construyen una aproximación de la matriz inversa del Hessiano $H^{-1}(x_k)$ utilizando únicamente la información de los gradientes obtenidos en pasos sucesivos.

El **Método del Gradiente Conjugado** es un algoritmo iterativo diseñado para encontrar el mínimo de una función objetivo cuadrática o, de forma equivalente, para resolver sistemas de ecuaciones lineales donde la matriz de coeficientes es simétrica y definida positiva.

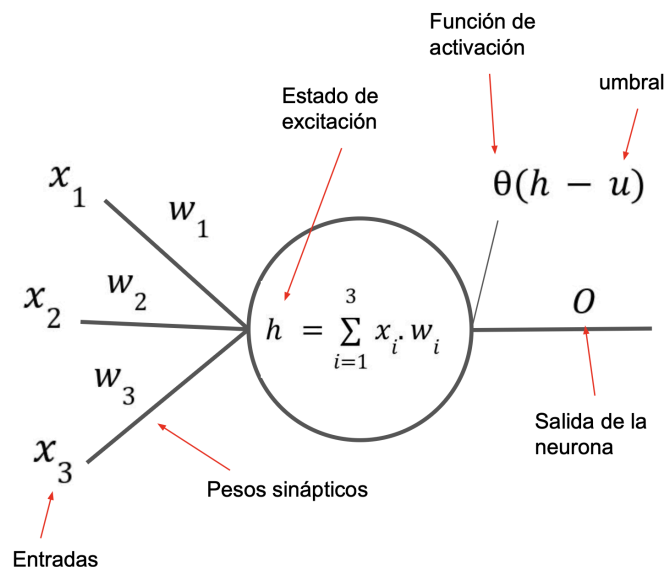
Ahora pasando a las funciones de error, tenemos los **métodos estocásticos**. La idea es que, como la función es muy larga y muy compleja, se hace una especie de estimación, como la media de todos los valores.. El **Gradiente Descendiente Estocástico** es uno de estos métodos. En términos matemáticos, el estimador del gradiente es el promedio de los gradientes individuales de cada observación del conjunto de entrenamiento, lo cual representa la dirección ideal para reducir el error total.

Unidad 8 - Aprendizaje Supervisado

Una **red neuronal** es un tipo de algoritmo de aprendizaje automático que está modelado según la estructura y función del cerebro humano. Consiste en una gran cantidad de nodos de procesamiento interconectados (neuronas) que trabajan juntos para procesar y analizar datos. Cada neurona recibe uno o más inputs, los procesa utilizando un conjunto de pesos aprendidos y produce una salida que se transmite a otras neuronas en la red.

El **aprendizaje supervisado** entrena un modelo con datos etiquetados, es decir cada ejemplo de entrada tiene una respuesta correcta conocida, para que aprenda a predecir esa respuesta en datos nuevos (por ejemplo, clasificar emails como spam o no spam).

El modelo se inspira en las neuronas del mundo real. Visto desde el modelo computacional:



La función de activación va a determinar si se dispara el estímulo o no.

El **hiperplano de separación** es una frontera geométrica fundamental en los modelos de clasificación, como el perceptrón, cuya función principal es dividir un conjunto de datos en dos clases distintas. Dependiendo de la cantidad de variables o dimensiones del problema, esta frontera toma diferentes formas: en un espacio

bidimensional se manifiesta como una recta, en tres dimensiones como un plano, y en espacios de mayor dimensionalidad se denomina formalmente hiperplano.

El mecanismo de clasificación opera mediante la **proyección de los datos** sobre este hiperplano. Matemáticamente, el modelo calcula una combinación lineal de las entradas (x) multiplicadas por sus respectivos pesos sinápticos (w), sumando además un término de umbral o bias (w_0). Si el resultado de esta proyección es positivo, el dato se asigna a una clase (generalmente representada como 1); si la proyección es negativa, se asigna a la clase opuesta (representada como -1).

Para obtener la clasificación final, se aplica una **función de activación (θ)**, usualmente de tipo escalón, sobre la suma ponderada de las entradas. Esta función devuelve el valor binario correspondiente según el lado del hiperplano en el que se encuentre el punto. En el contexto del aprendizaje, el objetivo del entrenamiento es encontrar, a través de un proceso iterativo, los valores óptimos de los pesos (w) que definan un hiperplano capaz de separar correctamente los datos o ajustar la recta de la mejor manera posible al conjunto de entrenamiento.

Lo que hicimos hasta ahora es **aprendizaje**. Refiere al proceso de entrenar un perceptrón sobre un conjunto de datos, con el objetivo de minimizar el error o la función de costo de la red sobre las entradas del conjunto de datos. La **generalización** refiere a la habilidad del perceptrón de desempeñarse correctamente sobre datos que no fueron alimentados durante el entrenamiento.

Es importante tener en cuenta el concepto de *garbage in garbage out*. Significa que si los datos de entrenamiento son de mala calidad, están mal etiquetados o son poco representativos, el modelo aprenderá patrones incorrectos y producirá predicciones igualmente malas, no importa qué tan sofisticada sea la arquitectura de la red.

El **Perceptrón Simple Lineal** representa una evolución del modelo básico diseñada para problemas donde el objetivo no es separar categorías, sino realizar una regresión o ajuste de datos. En este esquema, ya no se busca una frontera que divida clases, sino encontrar los valores de los pesos (w) que definan un hiperplano capaz de ajustarse lo mejor posible a la distribución de los puntos en el espacio.

La diferencia fundamental respecto al perceptrón tradicional radica en su función de activación, que en este caso es la función identidad. Esto implica que la salida de la neurona ya no es binaria (1 o -1), sino que puede tomar cualquier valor dentro de los números reales. Para entrenar este modelo, se define una función de error cuadrático que mide la discrepancia entre la salida real y la esperada; el objetivo del aprendizaje es minimizar este error acumulado.

Para lograr este ajuste, se aplica un procedimiento de optimización basado en el descenso del gradiente, donde los pesos se actualizan de manera iterativa. La regla de actualización (regla Delta) calcula la variación de cada peso en función de la tasa de aprendizaje, la entrada recibida y el error detectado. Una ventaja clave de este enfoque es que la fórmula de actualización es independiente de una función específica, siempre y cuando esta sea derivable. Por lo general, este tipo de entrenamiento se implementa en un formato online, actualizando los pesos de la red inmediatamente después de evaluar cada dato individual del conjunto de entrenamiento.

El **Perceptrón Simple No Lineal** se utiliza en escenarios donde no es posible ajustar los datos mediante una línea recta, requiriendo en su lugar el uso de una función de naturaleza no lineal. La diferencia fundamental con respecto al modelo lineal radica en el cambio de la función de activación, sustituyendo la identidad por funciones de tipo sigmoideo, como la tangente hiperbólica ($\tanh$) o la función logística.

Al utilizar estas funciones, la salida de la neurona queda confinada a rangos específicos (por ejemplo, de -1 a 1 para la tangente hiperbólica o de 0 a 1 para la logística), lo que generalmente hace necesaria una normalización de los datos si se desea predecir valores reales fuera de ese intervalo. Además, estas funciones suelen incorporar un parámetro (β) que permite modificar la forma de la curva para ajustarse mejor a la sensibilidad requerida por el problema.

En cuanto al aprendizaje, el modelo mantiene la misma fórmula de error cuadrático que el perceptrón lineal. Sin embargo, la regla de actualización de los pesos sinápticos se ajusta para incluir la derivada de la función de activación elegida. Esto permite que el entrenamiento siga siendo un proceso iterativo basado en el

descenso del gradiente, con la ventaja de que el sistema puede emplear cualquier función de activación siempre y cuando sea derivable.

El **Perceptrón Multicapa (MLP)** representa la evolución del modelo conexionista hacia estructuras capaces de resolver problemas que no son linealmente separables. Su concepto fundamental se basa en la combinación de múltiples neuronas organizadas en capas, donde la suma de sus funciones individuales permite representar y aproximar comportamientos complejos que un perceptrón simple no podría abordar por sí solo.

Estructuralmente, esta red se organiza en una jerarquía que incluye una capa de entrada, una o más capas ocultas y una capa de salida. El uso de múltiples neuronas en la última capa permite que el modelo no solo clasifique datos en diversos grupos, sino que también entregue un grado de probabilidad de pertenencia a cada clase. Esta capacidad está respaldada por el **Teorema de Aproximación Universal**, que garantiza que una red con al menos una capa oculta y funciones de activación no lineales (como la sigmoide o ReLU) puede aproximar cualquier función continua con un alto nivel de precisión.

El procesamiento de la información ocurre mediante un mecanismo llamado **feed-forward pass**, en el cual los datos viajan desde la entrada a través de las capas intermedias hasta generar un resultado final. Para que la red aprenda, se requiere minimizar una función de error ajustando los pesos sinápticos en la dirección opuesta al gradiente.

Es aquí donde toma relevancia el **backpropagation**, el algoritmo fundamental que permite calcular de manera eficiente cómo afecta cada peso al error total, aplicando la regla de la cadena para "repartir la culpa" del error entre todas las neuronas de la red y permitir su ajuste iterativo. Lo podemos ver:

$$\frac{\partial E}{\partial W_{11}} = \boxed{\frac{\partial E}{\partial O_1} \frac{\partial O_1}{\partial h_1}} \frac{\partial h_1}{\partial W_{11}}$$

$$\frac{\partial E}{\partial w_{11}^2} = \frac{\partial E}{\partial O_1} \frac{\partial O_1}{\partial h_1} \frac{\partial h_1}{\partial V_1^2} \frac{\partial V_1^2}{\partial h_1^2} \frac{\partial h_1^2}{\partial w_{11}^2}$$

$$\frac{\partial E}{\partial w_{11}^1} = \frac{\partial E}{\partial O_1} \frac{\partial O_1}{\partial h_1} \frac{\partial h_1}{\partial V_1^2} \frac{\partial V_1^2}{\partial h_1^2} \frac{\partial h_1^2}{\partial V_1^1} \frac{\partial V_1^1}{\partial w_{11}^1}$$

Para este algoritmo tenemos distintas variantes:

- Incremental/Online: La actualización de los pesos de la red se hace luego de calcular el Δw para un elemento del conjunto de datos.
- Mini Batch: La actualización de los pesos de la red se hace luego de calcular el Δw para un subconjunto de elementos del conjunto de datos.
- Batch: La actualización de los pesos de la red se hace luego de calcular el Δw para todos los elementos del conjunto de datos.

Para las métricas en el proceso de evaluación, tendremos las siguientes etapas:

1. Entrenar la red con el conjunto de entrenamiento.
2. Evaluar el método con el conjunto de entrenamiento.
3. Evaluar el método con el conjunto de prueba

La **Matriz de Confusión** es una tabla que permite evaluar el desempeño de un algoritmo de clasificación.

- Columna → cantidad de instancias en la clase dada por el método. (Predicción)
- Fila → cantidad de instancias en la clase verdadera. (Real/Actual)

		Prediction	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

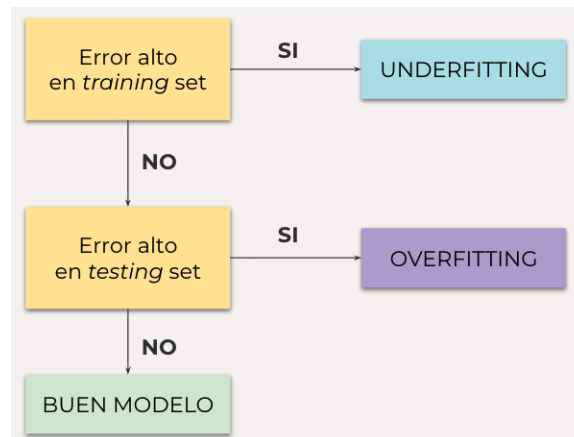
Notemos que vamos a querer maximizar la diagonal de la figura anterior, porque son los casos donde se detectan correctamente los positivos y negativos verdaderos.

Para evaluar la clasificación, existen una serie de **métricas estándar** que ya están definidas, por lo que debemos tener cuidado de no crear una propia y nombrarla exactamente igual. Algunas son:

- **Accuracy:** $\frac{TP+FN}{TP+TN+FN+FP}$ → puede presentar sesgos en casos donde hay pocas muestras, como por ejemplo en una enfermedad muy rara con pocos casos.
- **Precision:** $\frac{TP}{TP+FP}$ → mide la calidad del modelo, es decir la capacidad de evitar que las etiquetas negativas queden como positivas.
- **Recall:** $\frac{TP}{TP+FN}$ → informa sobre la cantidad de la clase positiva que el modelo identificó bien.
- **F1-Score:** $\frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$ → es una combinación de los anteriores.
- **Tasa de TP:** $\frac{TP}{TP+FN}$
- **Tasa de FP:** $\frac{FP}{FP+TN}$

El **underfitting** ocurre cuando un modelo es demasiado simple para capturar la estructura subyacente de los datos, como por ejemplo una línea recta que intenta explicar un patrón que claramente es ondulado. En este estado, el modelo presenta un sesgo (bias) alto, lo que significa que ni siquiera logra aprender bien del dataset de entrenamiento y, por lo tanto, tendrá un desempeño pobre tanto en la práctica como con datos nuevos.

El **overfitting** es el problema opuesto, donde el modelo es tan complejo que, en lugar de aprender el patrón general, termina "memorizando" el ruido y las fluctuaciones aleatorias del dataset de entrenamiento. La curva puede serpentear de forma errática para tocar cada punto, lo que resulta en un error de entrenamiento casi nulo pero una varianza (variance) muy alta. El gran riesgo aquí es que el modelo pierde su capacidad de generalización; cuando le presentes datos nuevos que no estaban en esa "foto" exacta, fallará estrepitosamente porque aprendió detalles irrelevantes en lugar de la tendencia real.



Una de las cosas a realizar antes de entrenar, es un procesamiento de los datos donde se suele realizar una **normalización**. Una técnica es future scaling, que consiste en escalar la data en un intervalo [a;b].

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}(b - a) + a$$

Otra opción es **estandarizar**, que consiste en tomar las variables del conjunto $P=\{X_1, \dots, X_p\}$ y calcular los siguientes estadísticos unidimensionales. Cada X_i tiene n registros.

MEDIA	DESVÍO ESTÁNDAR
$\bar{X}_i = \frac{1}{n} \sum_{j=1}^n X_i^j$	$s_i^2 = \frac{1}{n} \sum_{j=1}^n (X_i^j - \bar{X}_i)^2$
VARIABLE ESTANDARIZADA	
$\tilde{X}_i = \frac{X_i - \bar{X}_i}{s_i}$	

El problema que tiene el método del gradiente descendente es que la información que dispone es local. Esto es, porque la dirección de descenso es, justamente, la derivada de $E(W)$ respecto a cada uno de los pesos de las conexiones en el punto obtenido hasta ese momento. En el siguiente caso, nos puede suceder quedarnos en el mínimo al que está llegando el modelo, justamente por esta condición de contar con información local. El **momentum** es una técnica que toma la siguiente fórmula:

$$\Delta w_{ij}(t+1) = -\eta \frac{\partial E}{\partial w_{ij}} + \alpha \Delta w_{ij}(t)$$

Donde α está entre 0 y 1. Los valores comunes para α son 0.8 o 0.9. Vemos que esto le da el empujón que ya tiene de la iteración anterior a la actual, de allí su nombre.

El **Eta Adaptativo** es otra técnica que tiene la siguiente idea. Si, de forma consistente, $E()$ decrece en un momento del aprendizaje (por ejemplo, durante k iteraciones seguidas), quizás sería interesante incrementar η . Por otro lado, si $E()$ comienza a incrementarse en un momento del aprendizaje (por ejemplo, durante k' iteraciones seguidas), quizás sería interesante decrementar η para evitar divergencia.

Otro método es el **RMSProp**, que es una especie de Eta Adaptativo pero lo que hace es tener un η distinto para cada peso de la red. La fórmula es la siguiente, donde g_t es el gradiente en la época t .

$$g_t = \frac{\partial E}{\partial w_{ij}}$$

$$S_t = \gamma S_{t-1} + (1 - \gamma) g_t^2$$

$$\Delta w_{ij} = -\frac{\eta}{\sqrt{S_t + \epsilon}} * g_t$$

g_t^2 es el cuadrado aplicado miembro a miembro en el vector g_t

ϵ es un parámetro para evitar divisiones por 0 (en las primeras iteraciones)

El último método es el **Adam**, que resulta de la combinación de RMSProp y Momentum, y es muy utilizado en la práctica.

La **regularización** es un conjunto de técnicas diseñadas para reducir el error de test (o validación). Es decir, queremos disminuir el siguiente gap en el accuracy:



Busca bajar el error de generalización. No siempre acompañan el aprendizaje.

La **capacidad** de un modelo de Machine Learning es la habilidad o “potencial” de aproximar o alcanzar una variedad determinada de funciones. Algunas acciones que

permiten cambiar capacidad con la elección del modelo, su arquitectura o la cantidad de características que usamos como input.

La **Brecha de Generalización** es la distancia vertical entre la curva de error de entrenamiento y la de error de generalización. Si la brecha es pequeña, indica que el modelo se comportará de forma similar con datos nuevos que con los de entrenamiento. Si la brecha es grande, es el síntoma claro de Overfitting. El modelo "sabe mucho" de sus datos de práctica pero no entiende nada del mundo real. El **Punto Óptimo** de capacidad es el momento exacto en el que el modelo ha dejado de ser demasiado simple (bajo sesgo), pero aún no ha empezado a volverse "paranoico" con el ruido de los datos (baja varianza). Las técnicas de regularización ayudan en la parte "derecha" de la imagen previa.

El **Early Stopping** es la primera técnica de regularización, y es de las más simples y efectivas en el entrenamiento de redes neuronales. Su objetivo es detener el proceso de aprendizaje en el momento justo antes de que el modelo empiece a hacer overfitting. Para aplicarlo, se dividen los datos en Entrenamiento, Validación y Testeo.

Data Augmentation es un conjunto de técnicas utilizadas para incrementar artificialmente el tamaño y la diversidad de un conjunto de datos de entrenamiento sin necesidad de recolectar datos nuevos reales. Se logra aplicando transformaciones aleatorias a los datos ya recolectados, de modo que el modelo vea una versión ligeramente distinta de la misma información en cada época.

L2 Penalty Norm / Weight Decay es otra técnica de regularización fundamental. Actúa directamente sobre los pesos w de la red. Consiste en modificar la función de costo original (E) agregándole un término que "castiga" a los pesos grandes.

$$w_{nuevo} = w_{anterior}(1 - \eta\lambda) - \eta \frac{\partial E}{\partial w}$$

Donde λ (Lambda) es el coeficiente de regularización. Controla qué tan fuerte es la penalización.

Unidad 9 - Aprendizaje No Supervisado

En el **aprendizaje no supervisado**, la variable de respuesta no es información disponible. Puede resolver problemas menos definidos mediante Clustering, Asociación y Reducción de Dimensionalidad. Para saber si los resultados que obtenemos son significativos, se construyen modelos de predicción, que constan de estrategias para obtener características o patrones y sacar conclusiones.

En este caso, el modelo recibe una mezcla de datos no etiquetados sin instrucciones previas sobre qué representa cada uno. A través del análisis de patrones y características comunes, el modelo procesa la información para generar un output donde los datos similares aparecen agrupados en categorías lógicas.

El **clustering** consiste en agrupar observaciones de forma tal que el grado de similitud entre miembros de un mismo grupo sea lo más fuerte posible. Implica identificar similitudes entre los datos y asignarlos a un grupo (cluster).

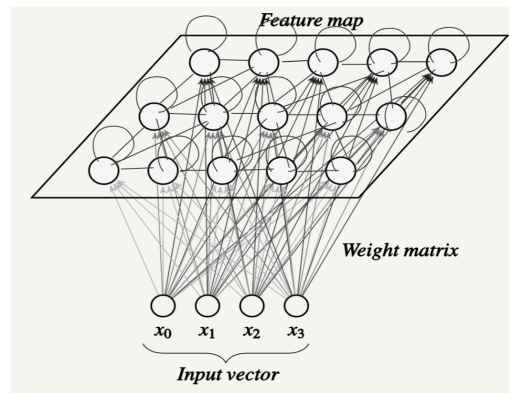
La **asociación** consiste en encontrar relaciones entre los atributos del conjunto de datos. Utiliza memorias asociativas, que consisten en el almacenamiento y recuperación de información por asociación con otros datos.

La **reducción de la dimensionalidad** proyecta el conjunto de datos en un espacio menor, dejando de lado las características menos relevantes. Sirve para, por ejemplo, identificar qué variables afectan más a cada país para luego invertir (educación, agricultura, inflación, etc).

Los problemas del aprendizaje supervisado que este nuevo esquema soluciona son:

- Etiquetas: Es costoso generar/obtener un gran conjunto de datos donde todos estén etiquetados. El Aprendizaje No Supervisado no necesita etiquetas.
- Dimensionalidad: Cuando tenemos muchas características (features) es costoso encontrar una buena aproximación. El Aprendizaje No Supervisado permite la reducción de la dimensionalidad.
- Outliers: La calidad del conjunto de datos es importante. Si se ignoran los outliers aprenderá de ellos y cometerá más errores. El Aprendizaje No Supervisado agrupa los outliers por un lado y el resto los datos por el otro

La **Red de Kohonen**, descubre por sí misma regularidades (patrones) en los datos de entrada durante el proceso de aprendizaje. La arquitectura de este modelo es la siguiente:



Las neuronas están conectadas con sí mismas positivamente y también con las neuronas vecinas ($R=n$), donde el vecindario es de radio R . Entonces:

- INPUT: elemento del training set
- OUTPUT: grilla/mapa (M)

A lo largo del tiempo (épocas), algunas unidades toman un nivel de activación mayor mientras que el nivel de las demás se anula. Entonces, las neuronas compiten unas con otras. El objetivo de este aprendizaje es agrupar los datos que se introducen en la red. El mapa nos mostrará un agrupamiento. Las informaciones similares son clasificadas formando parte de la misma categoría o grupo y deben activar la misma neurona de salida.

Algunas medidas de similitud (o funciones de propagación) que podemos tomar son:

Distancia Euclídea:

$$W_k = \arg \min_{1 \leq j \leq N} \{ \| X^p - W_j \| \}$$

Exponencial

$$W_k = \arg \min_{1 \leq j \leq N} \{ e^{-\| X^p - W_j \|^2} \}$$

Un **autovector** es un vector que, al ser sometido a una transformación lineal mediante una matriz (A), conserva su dirección original, aunque su magnitud puede verse alterada por un factor de escalamiento. Este factor de escala se denomina **autovalor** (o valor propio) y se representa comúnmente con la letra griega lambda

(λ). Se expresa mediante la ecuación $A\bar{v} = \lambda\bar{v}$, lo que indica que transformar el vector $\bar{v}$ con la matriz A es equivalente a simplemente multiplicarlo por el escalar λ .

Si las variables de un conjunto de datos están muy correlacionadas entonces posee información redundante. El objetivo del método de **Principal Component Analysis (PCA)** es eliminar la redundancia. La idea es transformar el conjunto de variables original en otro conjunto. Tendrá variables que son combinaciones lineales de las anteriores pero no están correlacionadas entre sí. Entonces, dadas p variables originales, se desean encontrar $q < p$ variables que sean combinaciones lineales de las p originales, recogiendo la mayor parte de la información o variabilidad de los datos. Si las variables originales no están correlacionadas, entonces no tiene sentido realizar un Análisis de Componentes Principales. Se hallan p componentes principales.

El autovalor λ_i se corresponde con la varianza de la componente i . Ordenando los autovalores de mayor a menor se logra reducir la dimensionalidad tomando los autovectores correspondientes a los primeros q autovalores, que son los que proveen mayor información (en términos de variabilidad).

Si alguna de las variables toma valores mayores a las demás entonces tendrá mayor varianza, pero no quiere decir que tenga mayor variabilidad. Cuando las escalas de medida de las variables son muy distintas, la maximización de la varianza depende decisivamente de estas escalas de medida y las variables con valores más grandes tendrán más peso en el análisis. Para evitar el problema se deben **estandarizar** las variables cuando calculamos las componentes principales. De esta manera las magnitudes de los valores numéricos de las variables originales serán comparables.

La **regla de Oja** es un algoritmo de aprendizaje no supervisado diseñado para permitir que una red neuronal aprenda a realizar el Análisis de Componentes Principales (PCA) de forma iterativa. Su función principal es encontrar la dirección de máxima varianza en un conjunto de datos, lo cual corresponde a la primera componente principal.

Oja demostró que una actualización simple ($\Delta w = \eta O x$) es inestable porque los pesos crecen infinitamente; por ello, propuso una fórmula que incluye un término de

normalización basado en desarrollos de Taylor para mantener el módulo del vector de pesos ($|w|$) acotado y tendiendo a 1. Tras varias iteraciones, el vector de pesos final (w_{final}) de la neurona converge hacia el autovector correspondiente al autovalor más grande de la matriz de correlación de los datos de entrada. En términos prácticos, esto significa que el peso se alinea con la dirección de mayor variación de la información. Utiliza la arquitectura de un perceptrón simple con una sola capa de salida.

La **Regla de Sanger** es una extensión de la regla de Oja. Mientras que esta última permite extraer únicamente la primera componente principal, la regla de Sanger utiliza un mecanismo de ortogonalización que permite a una red de múltiples neuronas extraer de forma ordenada las primeras k componentes principales (PCA) de los datos. El algoritmo garantiza que los pesos de las neuronas converjan hacia los autovectores de la matriz de covarianza de la entrada, logrando así una reducción de dimensionalidad que preserva la mayor cantidad de información posible (varianza) de manera eficiente y biológicamente inspirada.

El **Modelo de Hopfield** es un modelo que almacena información en la red y la asocia con patrones que tiene almacenados. Entonces, consiste en almacenar un conjunto de p patrones tal que cuando se presenta un nuevo patrón P_{in} , la red responda con el patrón almacenado P_{out} que más se parece a P_{in} .

- P_{in} : Patrón de estímulo.
- P_{out} : Patrón de respuesta

Es una red recurrente, ya que existe retroalimentación entre las neuronas, están todas conectadas entre sí (a diferencia de las redes feed-forward). No se conectan consigo mismas. Son binarias, y el conjunto de input y output permitido es $\{-1, 1\}$ (activo o inactivo) Todas las neuronas están en una sola capa de entrada y salida.

Entonces, el **Modelo de Hopfield** es una red neuronal que cumple las siguientes características:

- Cada neurona i es un perceptrón simple con la función de activación escalón $(1, -1)$.
- Cada par de neuronas (i, j) se conectan por el peso sináptico w_{ij} .

La red evoluciona hasta que S_i no se modifique más $\forall i = 1, \dots, n$. Cuando esto sucede, quiere decir que la red alcanzó un **estado estable**. El estado al que evoluciona se lo denomina **atractor**.

Veamos el siguiente detalle de convergencia, ya que Hopfield tiene varias limitaciones. Debemos analizar, si le damos al modelo un patrón de estímulo, ¿siempre llega a un patrón almacenado? Puede terminar convergiendo a algo que no queremos. El sistema es estable si el **término de crosstalk** es cero, es decir, cuando los patrones son ortogonales (el producto interno es cero). Esto no sucede siempre, por lo que esta es una de las limitaciones de esta red. El término de crosstalk es:

$$\frac{1}{N} \sum_j \sum_{\mu \neq \nu} \xi_i^\mu \xi_j^\mu \xi_j^\nu$$

Hopfield demostró que su red está asociada a una **función de Energía**, dada por:

$$H = -\frac{1}{2} \sum_{i,j} w_{ij} S_i S_j$$

Y que los mínimos locales de esta función son los patrones almacenados. Con esto, podemos tomar una propiedad central de una Función de Energía, que es que siempre decrece (o permanece constante) cuando el sistema evoluciona, y como los pesos sinápticos son simétricos:

$$H = - \sum_{j>i} w_{ij} S_i S_j$$

Las limitaciones de Hopfield son:

- El número máximo de patrones que puede almacenar es igual al 15 % del número de neuronas de la red.
- Los patrones deben ser “más o menos” ortogonales.
- La Función de Energía H puede tener otros mínimos locales que no son los patrones almacenados, llamados **estados espurios**. También son atractores, por lo que puede desembocar en ciclos.

Unidad 10 - Deep Learning

Deep Learning es un desarrollo de ciencias de la computación, pero a la vez muy interdisciplinario, con bases biomiméticas que utiliza una estructura jerárquica aglomerativa. En concreto es una tecnología revolucionaria que hoy está en el núcleo y centro de la frontera digital y que se caracteriza por haber logrado avance en las siguientes áreas:

- Computer Vision
- Natural Language Processing
- Speech Recognition

El área actual de Deep Learning está caracterizada por cuatro pilares:

- Deep, Profundidad: Múltiples Niveles de Composición.
- Feature Learning: Aprendizaje de la Representación.
- Datasets masivos y estandarizados.
- GPUs, Graphic Processing Units, hardware especializado para el procesamiento paralelo de multiplicación de matrices.

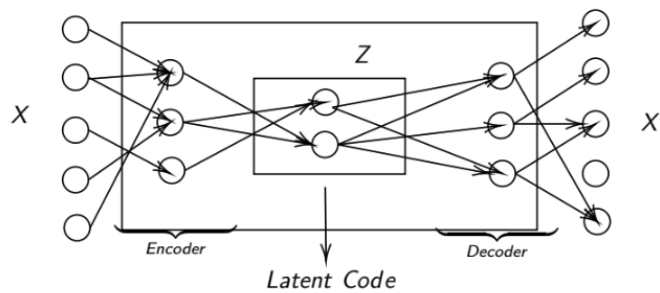
Los **autoencoders** son arquitecturas de redes neuronales no supervisadas cuyo objetivo principal es reducir la dimensionalidad. Consta de dos redes neuronales artificiales de perceptrones multicapa, donde la salida de la primera red se conecta con la entrada de la segunda. Esta tiene la distribución invertida de neuronas en las capas y como salida tiene la misma dimensión que la entrada de la primera red.

Formulación General

- $Z = f(X) = h(XW + b)$
- $X' = g(Z) = h(ZV + p)$

Esto debe satisfacerse de manera que $L(X, X') = ||X - X'||^2$.

La red es entrenada por cualquier método de aprendizaje que sirva para un Multi-Layer Perceptron, pero para cada patrón de entrada X se pone como salida esperada el mismo patrón X . La red por lo tanto aprende los pesos sinápticos que generan en la salida X' , el mismo valor presentado a la entrada.



Un **autencoder lineal** se conforma por perceptrones lineales. Una vez que aprende, minimiza:

$$J = ||X - ZV^T||$$

$$X \approx ZV^T$$

- X: La matriz de datos de $n \times d$. Hay n datos de dimensión d.
- Z: La salida de la capa interna del autoencoder, del código de $n \times k$.
- V: La matriz de pesos sinápticos asociada al decodificador de $k \times d$.

Lemma: PCA

La salida del código interno Z del autoencoder lineal son las salidas de las proyecciones de los datos en los componentes principales.

$$T_{PCA}(X) = X E = Z \quad (5)$$

Una de las herramientas que proveen los autoencoders es la **identificación de outliers**. Para esto, se lo entrena con el conjunto de entrenamiento. Luego se lo somete a nuevas muestras que no necesariamente se encuentran en el conjunto. Se mide con alguna medida la diferencia entre cada valor de entrada del autoencoder y los valores obtenidos en la salida.

El anglicanismo **denoising** hace referencia a la eliminación del ruido. Como el autoencoder genera una aproximación de la matriz de datos X en base a $X \approx X' = ZV^T$ entonces esto permite utilizarse para identificador de outliers y eliminación del ruido. La idea es que una vez aprendido un conjunto de datos, cuando aparecen nuevas muestras que pueden estar contaminadas con ruido, reemplazar la muestra ruidosa por la muestra obtenida en la salida del Decodificador. Para esto, en vez de entrenar con los elementos obtenidos directamente del conjunto de datos, se

modela el ruido, es decir, se lo caracteriza numéricamente (e.g. salt-and-pepper) o en general con una función de distribución de probabilidad (e.g. Gaussiano, Rayleigh) y se agrega a los datos X generando una nueva instancia $\tilde{X}$. Sin embargo, la salida que se pone en la red corresponde al dato X sin alterar.

Los **Contractive Autoencoder (CAE)** agregan un término regularizador a las funciones de costo, que le resta sensibilidad a la entrada, intentando aprender representaciones más simples y crudas de los datos.

$$\|J_h(X)\|_F^2 = \sum_{ij} \left(\frac{\partial h_j(X)}{\partial X_i} \right)^2$$

De esa manera por ejemplo, datos en dos dimensiones pueden reducirse a una dimensión, como una representación de una función de dos dimensiones, que pasa a una representación paramétrica (con un solo parámetro t).

Los **Sparse Autoencoder (SAE)** por otro lado, utilizan una arquitectura donde el espacio latente tiene más dimensiones de la entrada. Para lograr imponer restricciones que permitan adquirir información de la estructura de los datos, imponen un término regularizador que favorece la presencia de esparcidad, esto es, que un número de neuronas del espacio latente estén en 0 anuladas.

Una red neuronal que permite representar una función estocástica se denomina una **Red Neuronal Estocástica**. Para implementarlas se agrega un nodo, una capa donde se genera un número aleatorio pero de una forma que la red pueda seguir retropropagando el error. La clave para esto está en el **reparametrization trick** haciendo que el valor de z se calcule en función de $\bar{\mu}$ y de $\bar{\Sigma}$.

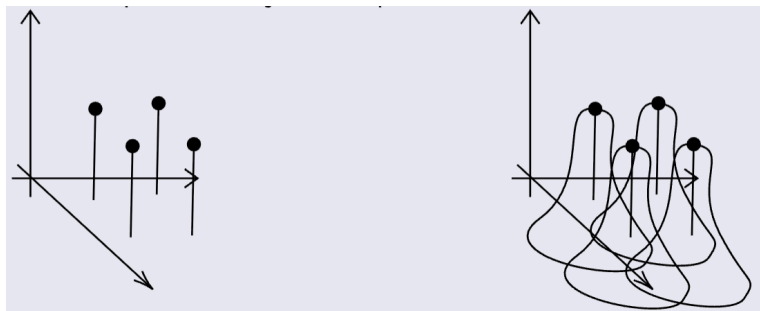
$$z = h(\bar{X}) = \epsilon \odot \bar{\Sigma}(\bar{X}) + \bar{\mu}(\bar{X})$$

El nodo estocástico recibe entonces los parámetros de una distribución de probabilidad, como por ejemplo, la media y el desvío estándar para una normal, para luego con esos parámetros, samplear un valor r que es utilizado con el truco de la reparametrización, para mostrar con esa distribución que tiene esos parámetros. El efecto final, es que la red se convierte en un mapeo estocástico, entre x e y .

Veamos cómo podemos, utilizando una red neuronal, implementar un mecanismo generativo causal, que me permita obtener muestras que comparten características representativas de un conjunto de datos. Esto va a ser posible gracias al **Algoritmo del Autoencoder Generativo**:

1. Utilizamos un Autoencoder para codificar en el Espacio Latente todos los patrones.
2. Luego dejamos de lado el Encoder, para movernos dentro de un vector de representación.
3. Generamos muestras sucesivas especificando de manera directa los valores de z_1 y z_2 .
4. Para cada tupla de valores de z_i obtenemos una nueva muestra generada por el Decodificador.

Ahora, vamos a darle una estructura estadística al espacio latente. Es decir, en vez de ver la salida que se obtiene para un único punto de z , ver las salidas que se obtienen para un conjunto de puntos.



Para solucionar esto, lo que vamos a hacer es darle estructura al espacio latente. Vamos a tomar las ideas de los **Variational Autoencoders (VAE)**, que es la convergencia de dos cuestiones:

- Una estimación variacional de un modelo generativo[14, 9, 6, 11].
- Una solución a esa estimación basada en dos redes neuronales feedforward acopladas, un Encoder y un Decoder, que conforman un Autoencoder.

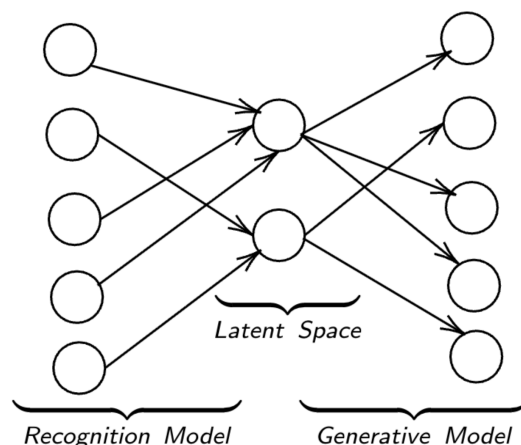
Los **métodos variacionales** son estrategias de la física estadística que se basa en establecer una función de costo la cual es mínima en la solución verdadera hipotética, pero que al plantear soluciones alternativas, generalmente paramétricas, es posible minimizando la función evaluar y encontrar una solución posible cada vez más buena.

La **inferencia variacional** intenta resolver el problema de estimar una función de densidad de probabilidad (PDF) aproximándola con funciones conocidas, y optimizando los parámetros de esa función mediante la solución de un problema de optimización.

Entonces ahora el objetivo es tratar de aproximar $p(z|x)$ con una $q(z|x)$, siendo $q(z)$ una función que queremos (e.g. una Gaussiana) y controlamos. Vamos a hacer una optimización de los parámetros de las funciones que nosotros ponemos para que aproximen a la de los datos que existen por detrás. Esta es la idea principal del **Autoencoder Variacional**. Entonces:

- $p(z|x)$: es la pdf de los datos que desconocemos, y que la vamos a aproximar por $q(z|x)$.
- $q(z|x)$: es la pdf que vamos a plantear (a gusto) para aproximar $p(z|x)$.
- $q(z)$: vamos a asumir que los datos tienen una distribución normal alrededor de cada z .
- $p(z)$: vamos a asumir que efectivamente probabilísticamente cada x mapea multidimensionalmente normal a z . Que sea normal es una restricción sobre la $p(\cdot)$. Podría ser otra.

El resultado entonces es el siguiente:



El modelo de reconocimiento se encarga justamente de reconocer la x y generar la cuestión paramétrica de cuál es la media y cuál es la varianza donde ese x va a parar dentro del espacio latente, lo que va a ir cambiando. Con esto, la salida empieza a ser como una mezcla. Se empiezan a generar regiones dentro del espacio latente que tienen un significado asociado a los datos.

El **Parametrization Trick** de la capa estocástica de la red, permite poder retropropagar el error manejando esa capa que tiene una entrada extra que es una variable aleatoria. No depende de la salida del decoder ($\bar{X}$), sino que depende de variables internas (Σ y μ) que llegan a la capa latente. Las dos salidas del encoder, actúan como entradas a la capa "z" donde está el truco de la parametrización, que a su vez, actúa como un perceptrón lineal con función de activación identidad.

Entonces:

1. Hay dos gradientes a retropropagar, uno es el de reconstrucción y el otro de regularización.
2. Los pesos del decoder se actualizan exclusivamente con la contribución del gradiente de reconstrucción. Esto es igual al caso de un perceptrón multicapa.
3. Los gradientes de reconstrucción del encoder se obtienen multiplicando el gradiente que viene del decoder por la derivada de la función del truco de la reparametrización respecto de media y la varianza.
4. Los gradientes del término de regularización del encoder se obtienen de la derivada de la divergencia KL respecto de la media y la varianza.
5. Se suman estos valores anteriores en cada paso del encoder.

Para poder entender **redes convolucionales** es importante primero definir y entender la operación de **convolución**. Es una operación lineal que se da sobre una señal, es decir una función unidimensional en el tiempo. Suponiendo que $x(t)$ es una señal unidimensional en función del tiempo y $w(t)$ es un núcleo de convolución:

- $s(t) = \int x(k)w(t-k)dk$
- $s(t) = (x * w)(t) = \langle x(k), w^*(t-k) \rangle$, donde $*$ es el conjugado.
- (Correlación $s(t) = \int x(k)w(t+k)dk$ (+ positivo))
- Discreta: $s[n] = \sum_k x[k]w[n-k]$

La convolución matemática cumple un rol primordial en el procesamiento de señales digitales. Por ejemplo, cualquier filtro digital lineal puede representarse por una igualdad basada en convoluciones discretas.

Una manera interesante de entender la convolución es verla como una extensión del promedio ponderado en una señal. Imaginemos que hay una señal, una función

donde el eje X es el tiempo t . Discretamente, para cada punto de t hay un valor en la señal, como podría ser el precio de bitcoin. Si queremos ver la tendencia de la información, sin perdernos en las variaciones diarias, lo que podemos hacer es para cada día, reemplazar el dato concreto del precio, por el promedio de los 5 días anteriores, hoy, y los 4 días posteriores. Eso va a suavizar la señal de precios, y nos va a permitir ver la tendencia sin importar tanto las variaciones diarias.

El proceso de convolución requiere tener un cuidado especial con los bordes. Como el soporte de la señal es finito y requiere alinearse con el núcleo, hay que manejar que pasa en los bordes, en relación a cómo se alinean la señal y el núcleo. Debemos que aplicar un **padding**, y hay tres maneras de realizarlo:

- **Valid:** implica alinear los valores del kernel con los valores existentes de la señal. Al estar haciendo un promedio, esto implica que el tamaño del resultado de esta operación, tiene menos cantidad de elementos (se reduce el tamaño del soporte en la salida).
- **Same:** corresponde a la situación donde se paddea, se agregan, a la señal original con dos ceros, uno de cada lado, y con eso permite que la señal de salida tenga exactamente la misma longitud que la de entrada, manteniendo el tamaño (por eso same).
- **Full:** implica agregar dos ceros de padding de cada lado, para que la señal generada incremente su tamaño.

Las **redes convolucionales** están basadas alrededor de la operación de convolución, y son generalmente aplicadas a procesamiento de imágenes. La idea es modificar una MLP de forma que realice una operación de convolución, de una capa a la otra. Se quiere imitar la misma idea de lo que se observa en la corteza visual, mediante la utilización de la convolución.

La **capa de convolución** es central, y sirve entenderla como una operación similar a la del Perceptrón, pero donde los pesos utilizados son compartidos espacialmente. Los diferentes componentes se pueden visualizar en la siguiente figura y ecuación. La operación en cuestión es de la capa q a la capa $q + 1$.

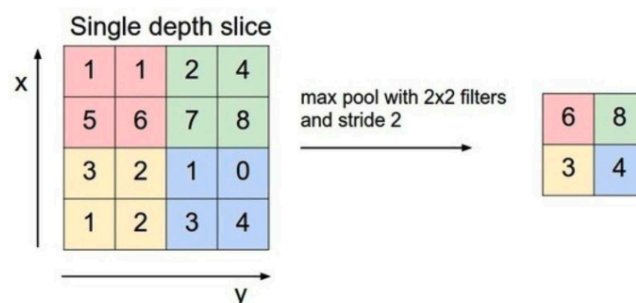
Al ser una operación de convolución, aparecen dos hiperparámetros nuevos, que son el **stride**, que es el paso del salto espacial que se va haciendo al pasar de un pixel al

siguiente (y calcular el promedio ponderado de todos los de alrededor). Y el segundo, por supuesto, es los **efectos de bordes** y cómo manejar el padding.

La **Capa de Pooling** está cableada, es decir no tiene ningún parámetro libre entrenable. Sirve para capturar invariantes traslacionales, y operan de manera similar a una operación de umbralización.

Diferentes regiones de la imagen van pasando por capas convolucionales, que son filtros, y enfatizan características particulares de la imagen. La capa de pooling permite que independientemente donde esté localizado ese punto enfatizado, eso se tenga en cuenta en capas sucesivas, y tenga relevancia, para afectar la salida de lo que la red genera. El resultado de esta capa es similar al efecto obtenido mediante el **downsampling**, o mediante la reducción de la resolución de una imagen.

La operación de pooling generalmente captura una región de la imagen y la sintetiza en un único píxel, normalmente eligiendo el mayor, conformando el **MaxPooling**, tal como se ve en la siguiente imagen. Esta invarianza traslacional puede extenderse a hacer otras características invariantes, dependiendo como se agrupen los filtros y las capas de pooling subsiguientes.



Al final de la Red hay una (o más capas) que se encargan de hacer la clasificación final (si ese es el objetivo). Esta subred es tradicionalmente un perceptrón multicapa entrenado para hacer una clasificación multiclase de la imagen de la entrada. En redes convolucionales es muy común utilizar la función de activación **Softmax**:

$$\sigma(\vec{V})_i = \frac{e^{V_i}}{\sum_{j=1}^K e^{V_j}}$$

con V_i como la salida lineal de la neurona de la capa final. Esta función actúa también como un normalizador de la salida, enfatizando las diferencias para dar como resultado una única salida ganadora, aquella que tenga el mayor valor de salida dado por la ecuación anterior. Al ser una normalización, la salida de todos los valores suma 1 por lo que se lo puede interpretar como una función de probabilidad.

¿Cómo impacta el cambio en la estructura de la red en las capas de convolución para la aplicación del algoritmo de **Backpropagation**? Lo que se busca siempre es mantener la identificación de la influencia de cada peso al momento de retropropagar los errores. Por eso es que en las capas de convolución, como los pesos son compartidos, es necesario acumular todas las variaciones a la hora de realizar la actualización de los pesos. Existen varias maneras de resolver esto.

Unidad 11 - Transformers

Los **transformers** fueron propuestos en el famoso trabajo **Attention Is All You Need**. Son redes neuronales utilizada para transformar una secuencia en otra basada en un mecanismo de **atención**. Tienen una arquitectura general que está determinada por un Encoder y un Decoder.

- **Encoder:** recibe la información de la cadena de entrada. Tiene dos componentes, el primero corresponde a una capa que implementa el mecanismo de atención, específicamente el mecanismo de **Self-Attention**, que relacionará los lexemas de la entrada entre sí. La segunda capa es un MLP. La salida pasa al Encoder subsiguiente del stack.
- **Decoder:** La salida final del último Encoder es la entrada a cada uno de los Decoders, y reciben una entrada adicional que viene de la salida hasta el momento para el primer Decoder, y de la salida del Decoder anterior para todos los otros. El Decoder tiene una capa de Self-Attention, luego una capa de Encoder-Decoder-Attention (que toma la salida de la capa anterior versus las entradas recibidas del Encoder) y finalmente un MLP. La salida final del último Decoder genera la salida actual, en este caso el token actual de traducción.

El **mecanismo de atención** es una capa interna que poseen todos los Encoders y Decoders, cuya función es tener en cuenta otras palabras de la secuencia mientras se está analizando una en particular. En el caso de los Encoders, esta capa le permite poner el foco en cualquier otra palabra de la entrada, en tanto que los Decoder únicamente pueden analizar palabras previas en la secuencia de salida. La capa de atención permite relacionar los lexemas del lenguaje entre sí. El cálculo de self-attention es el siguiente.

1. Por cada embedding x_i , se calculan 3 vectores llamados query, key y value multiplicando los embeddings por 3 matrices de pesos (Attention Weights) W_Q , W_K y W_V .
2. Se calcula un "score" por cada embedding que determina cuánta atención se le pone a ese embedding mientras se está procesando otro. Se hace el producto escalar entre q y el vector k de cada embedding.
3. Se dividen los scores por la raíz de la dimensión de vector k ($\sqrt{d_k}$)
4. Usando Softmax para los scores se obtienen probabilidades (p).

5. Se multiplica cada vector v por lo obtenido en la función de Softmax.
6. Se suman los vectores ponderados v' obteniendo el output de la capa de Self-Attention para cada token (en este ejemplo, para el primero)

Para implementarlo se calcula de forma matricial para un procesamiento más rápido:

$$\text{Attention}(K, Q, V) = \text{SoftMax}\left(\frac{QK^T}{\sqrt{d_k}}\right) * V = Z$$

Los **Embeddings** son los diferentes esquemas de codificación de los lexemas. El primer proceso consiste en dividir la entrada en componentes, denominado **tokenización**. Las palabras (o tokens) se transforman en números en base a un diccionario global de codificación. Luego, cada token es representado en un vector numérico de dimensión fija, capturando relaciones semánticas y sintácticas en función de su contexto en el lenguaje. Para ello se aplica un algoritmo de embedding.

El mecanismo de **Multi-Headed Attention** permite al Encoder enfocarse en distintas posiciones de la secuencia de manera simultánea, logrando así capturar múltiples tipos de relaciones semánticas y sintácticas al mismo tiempo.

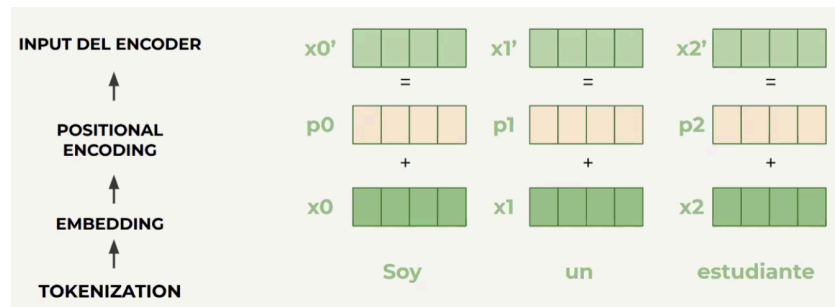
Una posible solución es agregar información a cada palabra sobre su posición en la oración. A esta información se la llama **Positional Encoding**, lo que es un conjunto de p_i valores que se le suma a cada embedding. Si bien hay diversas formas de abordar este inconveniente, una de ellas es utilizar las siguientes funciones sinusoidales para obtener el vector de posición del token i :

$$PE(pos, 2i) = \sin\left(\frac{pos}{1000^{\frac{2i}{d}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{1000^{\frac{2i}{d}}}\right)$$

donde

- pos es la posición del token en la secuencia.
- i es el índice del vector de posición.
- d es la dimensión de un embedding, la cual será la misma que un vector de posición p_i .



Unidad 12 - Redes Generativas Adversarias

Las redes **Generative Adversarial Networks (GAN)**, o redes adversariales generativas, surgen aproximadamente al mismo momento que los Autoencoders variacionales, abordando desde una perspectiva distinta la misma problemática: cómo generar muestras nuevas de un conjunto X . Fueron propuestas por un grupo de investigadores del núcleo de Deep Learning, entre los que se incluyen Ian Goodfellow y Yoshua Bengio.

Un **modelo generativo** describe la estructura de generación de un dataset X , en términos de su estructura probabilística. Sampleando de este modelo, es posible generar nuevas muestras X_j que serán probabilísticamente indistinguibles de las muestras originales X_i .

El problema por supuesto es estimar esa distribución. Las redes GAN buscan solucionar esto planteando una interacción entre dos redes. Una primera red es la responsable de discriminar entre las muestras reales pertenecientes al conjunto X , el dataset real, versus muestras artificiales. Este **Discriminador** entonces está sometido a recibir muestras de entrada y de manera supervisada aprender a decidir si cada una de las muestras produce el label de salida 1 o -1 indicando si pertenecen o no al conjunto real de muestras o son artificiales. Por otro lado, una segunda red llamada **Generador** va a tener el objetivo de producir las muestras falsas.

Como estamos tratando con PDFs, la función de costo que vamos a usar es la Binary Cross Entropy.

Binary Cross Entropy

$$H_p(q) = -\frac{1}{N} \sum_{i=1}^N \{(y_i) \log p(y_i) + (1 - y_i) \log(1 - p(y_i))\}$$

Donde $p(y_i)$ es la probabilidad de que el discriminador diga que la etiqueta es y_i .

Notemos que hay un conflicto acá presente. Una red quiere una cosa, y la otra red quiere exactamente la opuesta. El Discriminador quiere identificar con claridad cuáles son las muestras del dataset original, mientras que el Generador quiere generar una muestra que el Discriminador no pueda, justamente, discriminar. Este

conflicto se media mediante una función de costo que capture la película completa encuentre un balance.

$$\max_D H(y, D(x)) = \max_D \mathbb{E}_{x \sim P_{data}} \{\log(D(x))\} + \mathbb{E}_{z \sim P_z} \{\log(1 - D(G(z)))\}$$

donde lo que el Discriminador quiere es maximizar esa función para justamente acertar cuando corresponde hacerlo. Por otro lado, lo que el Generador quiere, desde su perspectiva es precisamente minimizar la ecuación previa: provocar que el Discriminador no identifique cuando las muestras han sido generadas por el Generador. Esto es, a la vez, optimizar:

$$\min_G \max_D V(D, G) = \min_G \max_D \left\{ \mathbb{E}_{x \sim P_{data}} \{\log(D(x))\} + \mathbb{E}_{z \sim P_z} \{\log(1 - D(G(z)))\} \right\}$$

El mínimo de esta ecuación se produce en un punto de equilibrio de Nash, donde se alcanza precisamente lo que se planteó al principio: el discriminador no tiene información para poder discriminar una muestra de X de las generadoras por el generador $G(z)$, sólo puede tirar la moneda, mientras que el Generador produce muestras nuevas que tienen la misma distribución estadística que la de los datos originales, es decir, $P_z \rightarrow P_{data}$.

El **colapso modal** es un fallo en el entrenamiento de las GANs que ocurre cuando el Generador descubre un tipo de dato específico que engaña con éxito al Discriminador, lo que provoca que se estanque y comience a producir únicamente copias o variaciones de ese mismo resultado. Como consecuencia, la red pierde su capacidad de diversificación y "colapsa" hacia un solo modo, ignorando por completo la variedad y riqueza del conjunto de datos original (por ejemplo, generando siempre el mismo número o la misma raza de perro).